Intitulé du cours: Technique des applications web

Framework django

Chapitre 2: Templates

Filière: 2éme année – bts web

Enseignante: Abid Arij

Année universitaire 2025/2026

Template en Django

🌿 C'est quoi un *Template* en Django ?

Un **template** est un **fichier HTML dynamique** utilisé par Django pour **afficher des pages web**.
Il mélange du **HTML classique** avec une **syntaxe spéciale Django** (appelée *template language*).

En résumé :

Le HTML définit la **structure** et le **design** de la page.
Django ajoute des **variables**, **conditions**, **boucles** et **héritage** pour générer des pages dynamiques.

À quoi sert un template ?

Les templates servent à **séparer la logique (Python)** de la **présentation (HTML)**.
C'est le principe **MVC (Model–View–Controller)** ou **MVT (Model–View–Template)** dans Django.

Élément	Rôle
Model	Gère les données (base de données)
View	Contient la logique et prépare les données
Template	Affiche les données dans la page web

Cela permet :

- D'éviter de mélanger HTML et code Python
- De **réutiliser** les pages plus facilement
- De **changer le design sans toucher au code Python**

À quoi sert un template ?

Les templates servent à **afficher les pages web** d'un site Django.
Ils permettent de **séparer la logique du code Python (dans les vues)** et **l'affichage HTML**.

Exemple :
Tu as une vue Django :

```
from django.shortcuts import render

def accueil(request):
    return render(request, 'accueil.html', {'nom': 'Arij'})
```

Et un template `accueil.html` :

```
<h1>Bienvenue {{ nom }} !</h1>
```

```
<h1>Bonjour {{ nom }}</h1>
```

Si la vue envoie `nom = "Arij"`, Django affiche :

Bonjour Arij

Pourquoi on a besoin de template ?

Sans templates, tu devrais **écrire le HTML directement dans le code Python**, ce qui serait :
difficile à maintenir,
peu lisible,
impossible à personnaliser facilement.

Les templates permettent :

- de **réutiliser** les mêmes structures HTML (en-têtes, pied de page, etc.)
- de **séparer la logique (Python)** et la **présentation (HTML)**
- de **générer des pages dynamiques** (avec boucles, conditions, etc.)

Un **template Django** = une page HTML “intelligente” qui peut afficher des **données venant du serveur (Python)**.

Comment fonctionne un template ?

1. Une **vue (view)** envoie des données à afficher.
2. Django **charge le template HTML**.
3. Le moteur de templates **remplace les variables** et **interprète les balises**.
4. La page finale (HTML complet) est envoyée au navigateur.

Types / Fonctionnalités des templates

I. Template simple :

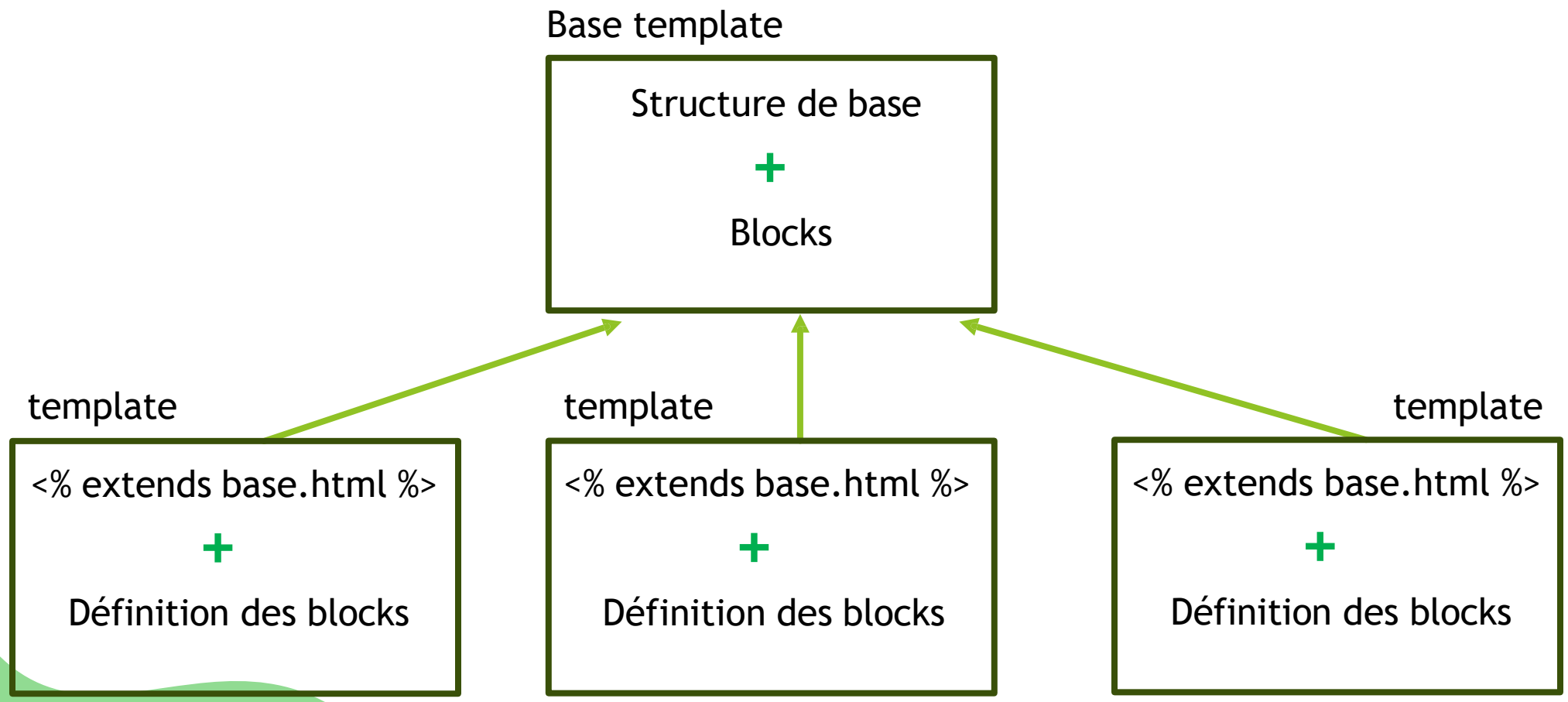
HTML

+

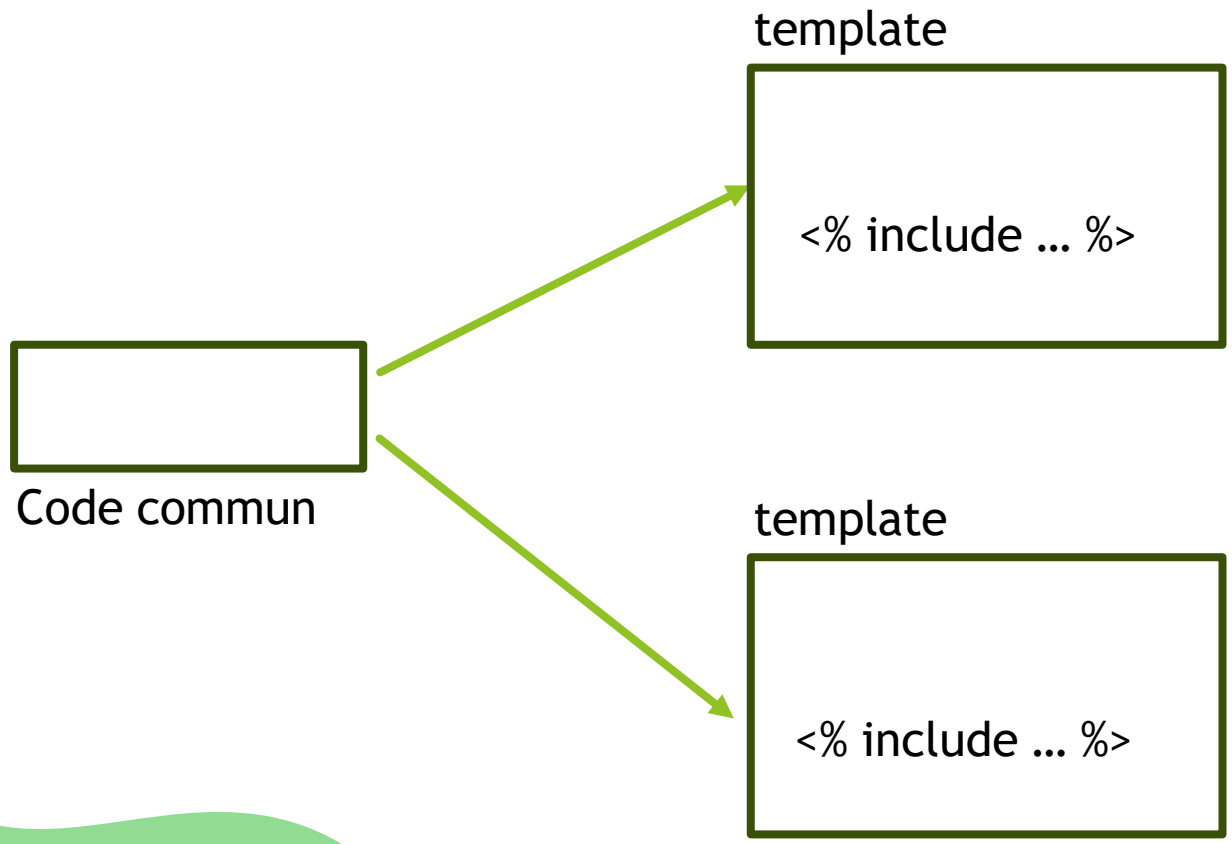
Django Template Langage:

1. **Template variables:** {{ context variable}}
2. **Template Tags:** <% for var in context vars %>
3. **Template filters:** {{ variable | filtername }}

2.Templates: héritage



3. Template: include



Syntaxe de base dans un template

Type	Syntaxe	Exemple
Variable	{{ variable }}	{{ nom_utilisateur }}
Balise / Tag	{% instruction %}	Boucle : {% for msg in messages %} ... {% endfor %}
Commentaire	{# commentaire #}	{# Ceci n'apparaît pas dans la page #}

{% if condition %} ... {% endif %} → condition
{% for item in liste %} ... {% endfor %} → boucle
{% block nom_du_bloc %} ... {% endblock %} → pour l'héritage

I. Template simple
Variables de template ({{ … }})

But : insérer dans le HTML la valeur d'objets passés depuis la vue (le *context*).
Exemples :

```
<p>Bonjour {{ user.first_name }} !</p>
<p>Prix : {{ product.price }}</p>
```

Les filtres en Django

Les **filtres** en Django permettent de **modifier ou transformer une variable** directement dans le template, sans toucher au Python. On les utilise avec le symbole | (pipe)..

Syntaxe d'un filtre: {{ variable|filtre }}

Filtres courants :

- date:"FORMAT" — formater une date ("d M Y", "SHORT_DATE_FORMAT", etc.)
- upper, lower — majuscule / minuscule
- title — première lettre majuscule de chaque mot
- truncatechars:10 — couper après N caractères
- truncatewords:10 — couper après N mots
- length — longueur d'une séquence
- join:"," — joindre les éléments d'une liste
- slice:"1:3" — tranche (comme Python)
- pluralize — ajoute le s si >1 : {{ count }} item{{ count|pluralize }}

Exemple:

```
{% comment "Exemple de filtres" %}
{% endcomment %}

<p>Nom en majuscules : {{ nom|upper }}</p>
<p>Nom avec majuscule initiale : {{ nom|title }}</p>
<p>Prix arrondi : {{ prix|floatformat:2 }} €</p>
<p>Liste de fruits : {{ fruits|join:", " }}</p>
<p>Statut actif : {{ actif|yesno:"Oui,Non" }}</p>
```

Si :

```
nom = "arij abid"
prix = 49.5
fruits = ["pomme","banane","orange"]
actif = True
```

Le rendu HTML sera :

Nom en majuscules : ARIJ ABID
Nom avec majuscule initiale : Arij Abid
Prix arrondi : 49.50 €
Liste de fruits : pomme, banane, orange
Statut actif : Oui

2.Templates: héritage

L’héritage permet de **définir une structure de base** (ex : header, footer, menu) et de **réutiliser** cette structure dans plusieurs pages.

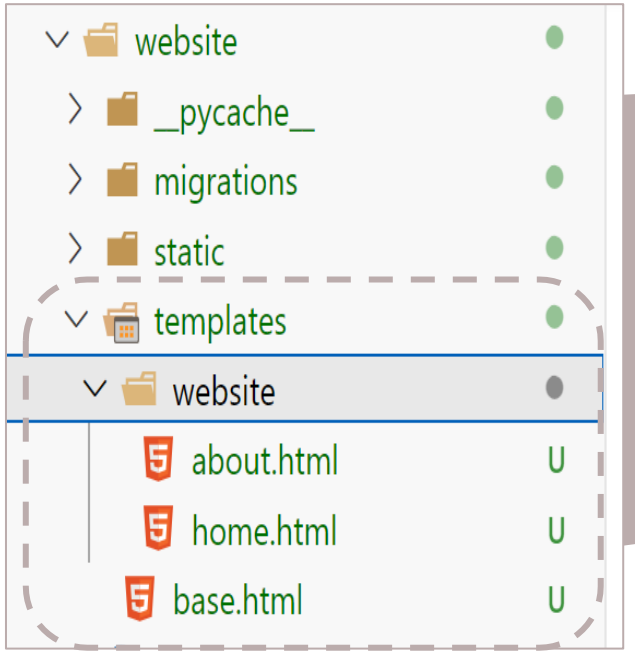
Templates: héritage (base.html)

- **Étape 1 — Créer un template parent base.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

    <title>{% block title %}{% endblock %}</title>

    </head>
<body>
    {% block content %}
    {% endblock %}
</body>
</html>
```



Templates: héritage (about.html)

Étape par Étape

Étape 2 — Créer un template enfant

about.html

```
{% extends "base.html" %}

{% block title %}about{% endblock %}

{% block content %}
    <h2>Meeting planner application  </h2>

{% endblock %}
```

Django **remplace les blocs** (`{% block ... %}`) du parent par ceux du fichier enfant.
Ainsi, toutes les pages ont le même design global (header/footer),
mais un contenu différent dans le bloc `content`.

website: home.html

Étape par Étape

Exemple:

```
{% extends "base.html" %}

{% block title %}HOME{% endblock %}

{% block content %}
    <h2>Welcome to the meeting planner</h2>
    <p>there are currently {{nbre_meeting}} meetings</p>

{% endblock %}
```

meetings.views

Étape par Étape

```
def detail(request, id):
    meeting = Meeting.objects.get(pk=id)
    return render(request, "meetings/detail.html", {"meeting": meeting})

def detail(request, id):
    meeting = get_object_or_404(Meeting, id)
    return render(request, "meetings/detail.html", {"meeting": meeting})
```

website: home.html

Étape par Étape

```
<h2>Meetings</h2>
  <ul>
        {% for meeting in meetings %}
              <li>
                    <a href="{% url 'detail' meeting.id %}">
                          {{ meeting.title }}
                    </a>
              </li>
        {% endfor %}
  </ul>
```

À quoi sert {% url %}

Le tag **{% url 'view_name' arg1 arg2 %}** sert à **générer automatiquement l'URL correspondant à une vue Django**.

- **Objectifs principaux :**

Ne pas coder en dur l'URL dans le template (évite les erreurs si l'URL change dans urls.py).
Créer des liens dynamiques pour chaque objet (ex : détails d'un meeting, profil utilisateur, article de blog…).
Rendre le site plus maintenable et sécurisé : si tu changes la route dans urls.py, tu n'as rien à changer dans tes templates.

website.views v3

Étape par Étape

```
def home_view(request):
    context={'meetings': Meeting.objects.all()}
    return render(request, "website/home.html",context=context)
```

website: home.html v3

Étape par Étape

```
<h2>Meetings</h2>
  <ul>
        {% for meeting in meetings %}
              <li>
                    <a href="{% url 'detail' meeting.id %}">
                          {{ meeting.title }}
                    </a>
              </li>
        {% endfor %}
  </ul>
  <a href="{% url 'rooms' %}">Rooms list</a>
```

Avantages de l'héritage de template

- Réutilisation du code HTML commun
- Modification du design global en un seul fichier
- Organisation claire du projet (un “squelette” principal)

3.Template: include

Inclut un autre template dans une page.

{% include 'menu.html’ %}

Utile pour les menus, footers ou sections réutilisées.

La balise **{% include 'fichier.html' %}** sert à **insérer le contenu d’un autre template dans la page en cours**.

- **En d’autres termes** : au lieu de copier-coller du code HTML identique dans plusieurs fichiers, tu peux **le mettre dans un template séparé** et l’inclure partout où tu en as besoin.

3.Template: include

Exemple concret

a) Template inclus — menu.html

```
<nav>
    <ul>
        <li><a href="/">Accueil</a></li>
        <li><a href="/apropos">À propos</a></li>
        <li><a href="/contact">Contact</a></li>
    </ul>
</nav>
```

b) Template principal — accueil.html

```
<h1>Bienvenue sur mon site</h1>

<!-- Inclusion du menu -->
{% include 'menu.html' %}

<p>Voici le contenu principal de la page d'accueil.</p>
```

En résumé I/2

Concept	Description	Exemple
Template	Page HTML dynamique	accueil.html
Variable	Donnée affichée	{{ nom }}
Balise (tag)	Contrôle logique	{% if %}, {% for %}, {% block %}
Include	Insère un autre template	{% include 'menu.html' %}
Extends	Héritage d'un template parent	{% extends 'base.html' %}
Block	Zone modifiable du parent	{% block content %}...{% endblock %}

En résumé 2/2

Balise	But
{% include 'file.html' %}	Insère un morceau de template (menu, footer…)
{% extends 'base.html' %}	Hérite d’un template parent (structure complète)

static en Django

Définition de static en Django

les fichiers **statiques** sont tous les fichiers **qui ne changent pas dynamiquement** lors de l'exécution du site.

Ils incluent :

- Les **CSS** (feuilles de style) → style.css
- Les **JS** (JavaScript) → script.js
- Les **images** → .png, .jpg, .svg, etc.
- Les **polices** → .woff, .ttf, etc.

En Django, le dossier **static** est l'endroit où tu mets tous ces fichiers pour que ton site puisse y accéder.

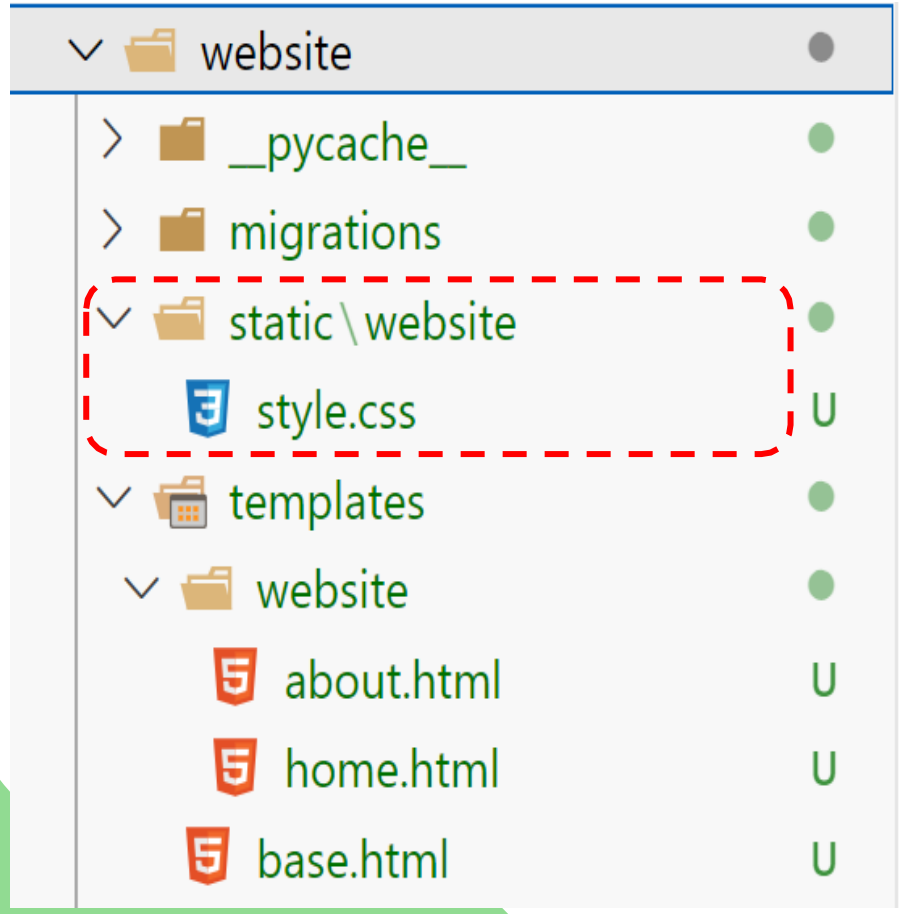
Organisation typique

Dans une app Django :

```
myproject/
├── myapp/
│   ├── static/
│   │   └── myapp/
│   │       ├── css/
│   │       │   └── style.css
│   │       ├── js/
│   │       │   └── script.js
│   │       └── images/
│   │           └── logo.png
│   ├── templates/
│   └── views.py
└── manage.py
```

Templates: static files

Étape par Étape




```
body {
    font-family: sans-serif;
    color: cornflowerblue;
    background-color: floralwhite;
}
```

Comment les utiliser dans un template HTML

Django propose le template tag {% static %} pour inclure un fichier statique.
Exemple :

```
{% load static %}  <!-- Obligatoire pour utiliser {% static %} -->

<!DOCTYPE html>
<html>
<head>
    <link rel="stylesheet" href="{% static 'myapp/css/style.css' %}">
    <title>Mon site</title>
</head>
<body>
    <img src="{% static 'myapp/images/logo.png' %}" alt="Logo">
    <script src="{% static 'myapp/js/script.js' %}"></script>
</body>
</html>
```

Templates: static files (settings.py)

Étape par Étape

```
# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/4.2/howto/static-files/

STATIC_URL = 'static/'
```

Templates: static files (base.html)

Étape par Étape

```
{% load static %}

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}{% endblock %}</title>
    <link rel="stylesheet"
          href="{% static 'website/style.css' %}">
</head>
<body>
    {% block content %}
    {% endblock %}
</body>
</html>
```

{% load static %} → charge la balise static pour être utilisée dans le template.
{% static 'chemin/vers/fichier' %} → génère l'URL correcte vers le fichier statique.

Pourquoi on utilise static ?

1. **Organisation** : Tu sais où sont tous les fichiers CSS, JS et images.
2. **Indépendance** : Les fichiers statiques ne dépendent pas des vues ou de la logique Python.
3. **Facilité de déploiement** : Django peut collecter tous les fichiers statiques pour les servir facilement en production.

Merci pour votre attention !